

Les méthodes d'optimisation en Machine Learning

Comment on entraîne les modèles — et *pourquoi* ça converge

5 juin 2026

Table des matières

1	Tout l'apprentissage est une optimisation	1
2	Concepts fondamentaux	1
2.1	Convexité	1
2.2	Conditions d'optimalité	2
2.3	Régularité : gradient Lipschitz, forte convexité	2
3	La descente de gradient	2
3.1	L'idée	2
3.2	Pourquoi ça optimise : le lemme de descente	2
3.3	Vitesses de convergence	3
3.4	Le pas α	3
4	Descente stochastique (SGD) et mini-batch	3
5	Accélérations : momentum, Nesterov, AdaGrad, RMSProp, Adam	3
5.1	Momentum	3
5.2	Taux adaptatifs	3
6	Méthode de Newton et IRLS	4
6.1	L'idée : utiliser la courbure	4
6.2	IRLS pour la régression logistique	4
7	Dualité de Lagrange et conditions KKT	4
8	Descente de coordonnées et méthodes proximales (Lasso)	5
9	Solutions fermées : les équations normales	5
10	Minimisation alternée (K-means, EM)	6
11	Descente de gradient <i>fonctionnelle</i> (boosting)	6
12	Problèmes spectraux (ACP, spectral clustering)	6
13	Tableau récapitulatif : pourquoi ça converge	7
14	Synthèse	7

1 Tout l'apprentissage est une optimisation

Entraîner un modèle, c'est presque toujours **minimiser une fonction de coût** $J(\theta)$ sur ses paramètres θ :

$$\hat{\theta} = \arg \min_{\theta} J(\theta).$$

Ce document est le « moteur » commun à tous les autres : il détaille les algorithmes qui résolvent ce problème et, surtout, *pourquoi* ils convergent. Le tableau ci-dessous relie chaque méthode à son optimiseur.

Méthode ML	Coût minimisé	Optimiseur
Régression linéaire	MSE (quadratique)	Équations normales / GD
Régression Ridge/Lasso	MSE + pénalité	Forme fermée / desc. coordonnées
Régression logistique	Entropie croisée	GD / Newton (IRLS)
SVM	Hinge + $\frac{1}{2} \ w\ ^2$	Dualité (QP, SMO)
Gradient Boosting	Perte arbitraire	Gradient <i>fonctionnel</i>
K-means	Inertie intra-cluster	Minimisation alternée
ACP / Spectral	Quotient de Rayleigh	Décomposition spectrale
Réseaux de neurones	Perte arbitraire	SGD + Adam (rétropropagation)

2 Concepts fondamentaux

2.1 Convexité

Définition 2.1. f est **convexe** si, pour tout x, y et $t \in [0, 1]$,

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y).$$

Si f est deux fois dérivable, c'est équivalent à $\nabla^2 f \succeq 0$ (Hessienne semi-définie positive).

La convexité « garantit » l'optimisation

Pour une fonction convexe, **tout minimum local est global**. Preuve : si $\nabla f(x^*) = 0$, la convexité donne pour tout y

$$f(y) \geq f(x^*) + \nabla f(x^*)^\top (y - x^*) = f(x^*).$$

Donc x^* est un minimum *global*. C'est pourquoi on insiste tant sur la convexité : régression linéaire/logistique, SVM, Lasso, Ridge ont tous un coût convexe $\Rightarrow$ pas de minima locaux parasites, l'optimiseur trouve la *vraie* solution.

2.2 Conditions d'optimalité

Pour un problème sans contrainte : $\nabla f(x^*) = 0$ (point stationnaire) et $\nabla^2 f(x^*) \succeq 0$ (minimum). Avec contraintes : conditions de **KKT** (section 7).

2.3 Régularité : gradient Lipschitz, forte convexité

On dit que ∇f est **L -Lipschitz** si $\|\nabla f(x) - \nabla f(y)\| \leq L \|x - y\|$ (la courbure est bornée par L). f est **μ -fortement convexe** si $f - \frac{\mu}{2} \|x\|^2$ reste convexe. Ces deux constantes contrôlent la **vitesse** de convergence.

3 La descente de gradient

3.1 L'idée

Le gradient $\nabla f(x)$ pointe dans la direction de plus forte *augmentation*. On avance donc dans la direction **opposée**, d'un pas $\alpha > 0$ (*learning rate*) :

$$x_{t+1} = x_t - \alpha \nabla f(x_t). \quad (1)$$

3.2 Pourquoi ça optimise : le lemme de descente

Lemme 3.1 (Descente). Si ∇f est L -Lipschitz, alors pour tout x et tout pas α ,

$$f(x - \alpha \nabla f(x)) \leq f(x) - \alpha \left(1 - \frac{L\alpha}{2}\right) \|\nabla f(x)\|^2.$$

Démonstration. La L -régularité implique l'inégalité quadratique $f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2$. On pose $y = x - \alpha \nabla f(x)$:

$$f(y) \leq f(x) - \alpha \|\nabla f(x)\|^2 + \frac{L\alpha^2}{2} \|\nabla f(x)\|^2 = f(x) - \alpha \left(1 - \frac{L\alpha}{2}\right) \|\nabla f(x)\|^2. \quad \square$$

Pourquoi ça optimise

Tant que $0 < \alpha < \frac{2}{L}$, le coefficient $\alpha(1 - \frac{L\alpha}{2})$ est **strictement positif** : chaque pas **diminue** f d'une quantité proportionnelle à $\|\nabla f\|^2$. La fonction décroît jusqu'à ce que $\nabla f = 0$. Avec le pas optimal $\alpha = 1/L$: $f(x_{t+1}) \leq f(x_t) - \frac{1}{2L} \|\nabla f(x_t)\|^2$. **Voilà pourquoi la descente de gradient optimise** : ce n'est pas une heuristique, c'est une décroissance *garantie*.

3.3 Vitesses de convergence

- f convexe, ∇ Lipschitz : $f(x_t) - f^* = O(1/t)$ (sous-linéaire).
- f **fortement** convexe : convergence **linéaire** (géométrique), $f(x_t) - f^* \leq c \rho^t$ avec $\rho < 1$.

Le **conditionnement** $\kappa = L/\mu$ gouverne la vitesse : grand κ (vallée allongée) $\Rightarrow$ convergence lente, d'où l'intérêt des accélérations.

3.4 Le pas α

Trop grand : divergence (on « saute » par-dessus le minimum). Trop petit : convergence lente. On le règle par recherche, planification (*decay*), ou recherche linéaire.

4 Descente stochastique (SGD) et mini-batch

Quand $J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta)$ (somme sur n exemples), calculer le gradient complet coûte $O(n)$ par pas. La **descente de gradient stochastique** (SGD) utilise un seul exemple (ou un *mini-batch* $\mathcal{B}$) tiré au hasard :

$$\theta_{t+1} = \theta_t - \alpha \nabla \ell_{i_t}(\theta_t) \quad \text{ou} \quad \theta_{t+1} = \theta_t - \frac{\alpha}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta_t).$$

Pourquoi ça optimise

Le gradient stochastique est un **estimateur sans biais** du vrai gradient : $\mathbb{E}[\nabla \ell_{i_t}(\theta)] = \nabla J(\theta)$. « En moyenne », on descend donc dans la bonne direction. Le bruit introduit une variance, qui oblige à réduire le pas ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$, conditions de Robbins-Monro) pour converger. **Pourquoi le préférer** : chaque pas est n fois moins cher, et le bruit aide même à échapper aux mauvais points-selles des réseaux profonds.

5 Accélérations : momentum, Nesterov, AdaGrad, RMSProp, Adam

5.1 Momentum

On accumule une « vitesse » (moyenne mobile des gradients) :

$$v_{t+1} = \beta v_t + \nabla f(x_t), \quad x_{t+1} = x_t - \alpha v_{t+1}.$$

Intuition

Dans une vallée allongée (grand κ), la GD oscille entre les parois. Le momentum **amortit ces oscillations** (les composantes opposées s'annulent) et **accumule de la vitesse** dans la direction constante du fond de vallée. **Nesterov** évalue le gradient « un pas en avant », ce qui anticipe et stabilise davantage.

5.2 Taux adaptatifs

AdaGrad divise le pas par la racine de la somme des carrés des gradients passés (pas plus petit pour les paramètres souvent mis à jour). **RMSProp** remplace cette somme par une moyenne mobile (évite que le pas s'éteigne). **Adam** combine momentum (moment m_t) et RMSProp (moment v_t) avec correction de biais :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f)^2, \quad x_{t+1} = x_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Pourquoi ça optimise

Ces méthodes **adaptent le pas par dimension** en fonction de la géométrie locale (l'histoire des gradients). Concrètement, elles « reconditionnent » le problème : grands pas dans les directions plates, petits pas dans les directions raides. C'est pourquoi Adam converge si bien sur les surfaces mal conditionnées des réseaux de neurones, sans réglage fin du pas.

6 Méthode de Newton et IRLS

6.1 L'idée : utiliser la courbure

La GD n'utilise que la pente (∇f). Newton utilise aussi la **courbure** (Hessienne $H = \nabla^2 f$). On minimise le **développement de Taylor d'ordre 2** autour de x_t :

$$f(x_t + \Delta) \approx f(x_t) + \nabla f(x_t)^\top \Delta + \frac{1}{2} \Delta^\top H \Delta,$$

dont le minimum (en annulant la dérivée en Δ : $\nabla f + H\Delta = 0$) est $\Delta = -H^{-1} \nabla f$. D'où

$$x_{t+1} = x_t - H^{-1} \nabla f(x_t). \quad (2)$$

Pourquoi ça optimise

Newton **saute directement au minimum de l'approximation quadratique locale**. Si f est exactement quadratique (ex. régression linéaire), il converge en **une seule itération**. Près de l'optimum, la convergence est **quadratique** (le nombre de décimales correctes double à chaque pas), bien plus rapide que la GD. Le prix : inverser une matrice $n \times n$ à chaque pas ($O(n^3)$), prohibitif en grande dimension.

6.2 IRLS pour la régression logistique

Appliquée à la log-perte logistique, la méthode de Newton devient l'**IRLS** (moindres carrés pondérés itérés) : avec $H = \frac{1}{m} X^T R X$ et $R = \text{diag}(p_i(1 - p_i))$,

$$w_{t+1} = w_t - (X^T R X)^{-1} X^T (p - y).$$

Chaque itération résout un problème de moindres carrés pondérés (d'où le nom). C'est l'optimiseur classique de la régression logistique sur données de taille modérée.

7 Dualité de Lagrange et conditions KKT

Pour un problème **sous contraintes** (le cas du SVM), on passe par la dualité. Soit

$$\min_x f(x) \quad \text{s.c.} \quad g_i(x) \leq 0.$$

Le **Lagrangien** est $\mathcal{L}(x, \lambda) = f(x) + \sum_i \lambda_i g_i(x)$, $\lambda_i \geq 0$. La **fonction duale** est $\mathfrak{g}(\lambda) = \inf_x \mathcal{L}(x, \lambda)$.

Théorème 7.1 (Dualité et KKT). *On a toujours $\mathfrak{g}(\lambda) \leq f(x)$ (**dualité faible**). Si f, g_i sont convexes et une condition de qualification (Slater) tient, alors **dualité forte** : $\max_\lambda \mathfrak{g}(\lambda) = \min_x f(x)$, et la solution vérifie les **conditions KKT** :*

$$\underbrace{\nabla_x \mathcal{L} = 0}_{\text{stationnarité}}, \quad \underbrace{g_i(x) \leq 0, \lambda_i \geq 0}_{\text{faisabilité}}, \quad \underbrace{\lambda_i g_i(x) = 0}_{\text{complémentarité}}.$$

Pourquoi résoudre le dual du SVM ?

- Le dual ne fait apparaître les données que par des **produits scalaires** $\Rightarrow$ **kernel trick** (non-linéarité à coût fini).
- Les contraintes deviennent de simples **bornes** $0 \leq \alpha_i \leq C$ (problème quadratique « boîte », facile).
- La **complémentarité** $\lambda_i g_i = 0$ explique la **parcimonie** : la plupart des $\alpha_i = 0$, seuls les vecteurs de support comptent.

L'algorithme **SMO** (Sequential Minimal Optimization) optimise deux α_i à la fois (le minimum permis par la contrainte $\sum \alpha_i y_i = 0$), avec une solution analytique à chaque étape.

8 Descente de coordonnées et méthodes proximales (Lasso)

La pénalité L_1 du Lasso n'est pas dérivable en 0 : la GD classique ne s'applique pas directement. On utilise la **descente de coordonnées** : minimiser le coût une variable à la fois.

Proposition 8.1 (Seuillage doux). *Le sous-problème à une coordonnée, $\min_\beta \frac{1}{2}(z - \beta)^2 + \lambda|\beta|$, a pour solution l'**opérateur de seuillage doux***

$$\beta^* = S(z, \lambda) = \text{sign}(z) (|z| - \lambda)_+.$$

Démonstration. La condition d'optimalité (sous-gradient) est $\beta - z + \lambda \partial|\beta| = 0$. Pour $\beta > 0$: $\beta = z - \lambda$ (valide si $z > \lambda$). Pour $\beta < 0$: $\beta = z + \lambda$ (valide si $z < -\lambda$). Pour $|z| \leq \lambda$: $0 \in [\beta - z - \lambda, \beta - z + \lambda]$ en $\beta = 0$. On regroupe en $\text{sign}(z)(|z| - \lambda)_+$. $\square$

Pourquoi ça optimise

Comme le coût = partie lisse (séparable presque) + pénalité **séparable** $\sum_j |\beta_j|$, minimiser cycliquement chaque coordonnée **converge vers l'optimum global** (résultat de Tseng pour les fonctions convexes « lisse + séparable »). Le seuillage doux **annule** les petites coordonnées $\Rightarrow$ c'est le mécanisme exact de la **parcimonie** du Lasso. Les méthodes **proximales** (ISTA/FISTA) généralisent l'idée : pas de gradient sur la partie lisse + opérateur proximal sur la partie non lisse.

9 Solutions fermées : les équations normales

Certains problèmes se résolvent **exactement**, sans itérer. Pour la régression linéaire, $J(\beta) = \|y - X\beta\|^2$ est une **quadratique convexe** ; annuler le gradient donne les équations normales :

$$\nabla J = -2X^\top y + 2X^\top X\beta = 0 \Rightarrow \hat{\beta} = (X^\top X)^{-1}X^\top y.$$

Pour le Ridge : $\hat{\beta} = (X^\top X + \lambda I)^{-1}X^\top y$.

Pourquoi ça optimise

Pour une **quadratique convexe**, le point stationnaire *est* le minimum global (Hessienne $2X^\top X \succeq 0$ constante). Pas besoin d'itérer : une seule résolution de système linéaire suffit. C'est l'optimisation dans son cas le plus simple. (En très grande dimension, on préfère quand même la GD, car inverser $X^\top X$ coûte $O(d^3)$.)

10 Minimisation alternée (K-means, EM)

Quand le coût dépend de **deux groupes** de variables, on les optimise **tour à tour**. K-means alterne assignation (à centroïdes fixés) et mise à jour des centroïdes (à assignation fixée) ; l'algorithme EM alterne « Expectation » et « Maximization ».

Pourquoi ça optimise

Chaque étape minimise (ou ne dégrade pas) **le même coût** J par rapport à un bloc de variables. Donc la suite $J^{(0)} \geq J^{(1)} \geq \dots \geq 0$ est **décroissante et minorée** $\Rightarrow$ elle **converge**. Pour K-means, le nombre de partitions étant fini, l'algorithme **termine** en un point fixe. *Attention* : minimisation alternée $\neq$ minimum global (le coût de K-means est non convexe) $\Rightarrow$ d'où K -means++ et les multi-démarrages.

11 Descente de gradient *fonctionnelle* (boosting)

Le Gradient Boosting fait une descente de gradient non pas dans l'espace des paramètres, mais dans l'**espace des fonctions**. On traite les valeurs $F(x_i)$ comme des variables et on calcule les **pseudo-résidus** :

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}, \quad F_m = F_{m-1} + \nu \gamma_m h_m,$$

où h_m est un arbre ajustant les $r_i^{(m)}$.

Pourquoi ça optimise

$-\frac{\partial L}{\partial F}$ est le **gradient négatif** du risque par rapport à la fonction : ajouter un modèle qui l'approxime fait **décroître la perte**, exactement comme un pas de descente de gradient. Pour la perte quadratique, $r_i = y_i - F_{m-1}(x_i)$ (le résidu) : booster = corriger itérativement l'erreur restante. Le shrinkage ν joue le rôle du *learning rate*.

12 Problèmes spectraux (ACP, spectral clustering)

Certaines « optimisations » se résolvent par **algèbre linéaire**, pas par itérations de gradient. L'ACP maximise un **quotient de Rayleigh** :

$$\max_{\|u\|=1} u^T \Sigma u.$$

Pourquoi ça optimise

Par le théorème de **Courant–Fischer** (Rayleigh–Ritz), le maximum de $u^T \Sigma u$ sous $\|u\| = 1$ vaut la **plus grande valeur propre** $\lambda_{\max}$, atteint au **vecteur propre** associé. La preuve passe par le Lagrangien : $\Sigma u = \lambda u$. Donc *calculer les vecteurs propres est résoudre l'optimisation* — globalement et exactement, en une fois. Pour k directions (inégalité de Ky Fan), on prend les k plus grandes. Le **spectral clustering** fait de même avec les *plus petites* valeurs propres du Laplacien (relaxation du problème de coupe).

13 Tableau récapitulatif : pourquoi ça converge

Optimiseur	Utilisé par	Pourquoi ça optimise
Descente de gradient	Lin./logistique, SVM, NN	Lemme de descente : f décroît si $\alpha < 2/L$
SGD / Adam	Grands jeux, NN	Gradient sans biais ; pas adaptatif reconditionne
Newton / IRLS	Logistique	Min. exacte de la quadratique locale (conv. quadratique)
Dualité / KKT (SMO)	SVM	Dualité forte (convexe) ; complémentarité $\Rightarrow$ parcimonie
Desc. coordonnées / prox	Lasso	Convexe lisse + séparable ; seuillage doux
Équations normales	Lin./Ridge	Quadratique convexe : stationnaire = global
Min. alternée	K-means, EM	Coût décroissant et minoré $\Rightarrow$ converge
Gradient fonctionnel	Boosting	Pseudo-résidu = $-\nabla$ du risque
Décomposition spectrale	ACP, spectral	Rayleigh–Ritz : vecteurs propres = optimum

14 Synthèse

À retenir

- **Apprendre** = minimiser un coût. La **convexité** garantit que le minimum trouvé est global.
- La **descente de gradient** optimise grâce au **lemme de descente** : pour $\alpha < 2/L$, le coût décroît strictement. SGD/Adam la rendent praticable et rapide à grande échelle.
- **Newton** exploite la courbure (convergence quadratique) ; **IRLS** en est la version logistique.
- La **dualité/KKT** transforme le SVM en QP à bornes, ouvre le kernel trick et explique la parcimonie.
- **Descente de coordonnées + seuillage doux** résout le Lasso et produit la sparsité ; les **équations normales** donnent une solution fermée aux problèmes quadratiques.
- **Minimisation alternée** (K-means/EM) converge par décroissance monotone ; le **gradient fonctionnel** fonde le boosting ; les **méthodes spectrales** résolvent exactement les quotients de Rayleigh (ACP, spectral).